Lab 02 Tasks

Contents

Submission Guidelines:

Repository Architecture

Your root directory must exactly follow this structural blueprint. Avoid dumping all files into the main folder; use clear, logical nesting.

StudentName_Medical_Imaging_Portfolio/

```
|
├── Task_1_Chest_XRay/
|   ├── data/                   # The specific X-ray images used for testing
|   ├── output/                 # Screenshots/saved images of your enhanced results
|   ├── xray_enhancement.ipynb  # Your Python script
|   └── README.md               # Brief explanation of your processing pipeline
|
├── Task_2_Cardiac_Fusion/
|   ├── data/                   # The specific CT and MRI matched pairs used
|   ├── output/                 # Saved fused heatmaps and comparison charts
|   ├── modal_fusion.ipynb      # Your Python script
|   └── README.md               # Your fusion weighting logic and analysis
|
├── Task_3_Echo_Analysis/
|   ├── data/                   # The short ultrasound .mp4 snippet used
|   ├── output/                 # Screenshots of the side-by-side video pipeline
|   ├── realtime_echo.ipynb     # Your Python script
|   └── README.md               # Instructions on how to run your video loop
|
├── requirements.txt            # Global library dependencies (OpenCV, NumPy, etc.)
└── README.md                   # Main repository overview and setup instructions
```

Folder Contents & Dependencies

- **Dataset Handling (The 100MB Rule):** GitHub has a strict file size limit. **Do not** attempt to upload the entire 1GB+ Stanford EchoNet or Kaggle datasets to your repository. Instead, create a data/ folder inside each task and upload *only the specific sample images or short video clips* your script actually uses.
- **Relative File Paths:** Your Python scripts must use relative paths (e.g., cv2.imread('data/sample_xray.png')). Do not use absolute local paths (like C:/Users/Student/Desktop/image.png).
- **Dependency Management:** You must include a requirements.txt file in the root directory listing all necessary Python packages (e.g., opencv-python==4.8.0, numpy==1.26.0). This allows the reviewer to instantly rebuild your environment.

Task 1: Diagnostic Enhancement of Chest X-Rays

The Scenario:

You are a junior computational researcher at a pulmonary imaging lab. The lab has acquired a massive dataset of chest X-rays to train a pneumonia-detection algorithm. However, many of the raw X-rays suffer from severe underexposure, making the lung cavities and potential fluid build-up nearly invisible. Your task is to mathematically process these single-channel matrices to maximize structural visibility before they are handed off to the diagnostic doctors.

The Operations:

- **Load and Display:** Load a sample grayscale image from the dataset.
- **Contrast Enhancement:** Apply Histogram Equalization to the X-ray matrix to forcefully redistribute the pixel intensities. Compare the equalized matrix with the raw image to document the newly visible lung structures.
- **Color Mapping:** Convert the equalized grayscale image into a false-color heatmap using OpenCV's COLORMAP_JET. Document how mapping intensity to a color spectrum helps the human eye detect subtle fluid boundaries.
- **Color Balance:** Simulate a lighting correction by applying a mathematical color balance shift to neutralize any artificial color casts in the visualization.
- **Color Filtering (Thresholding):** Because dense tissue (like bone or dense fluid) reflects specific high-intensity values, apply a strict mathematical threshold to mask out everything except the densest tissues.
- **Logarithmic Transformation:** Apply a logarithmic curve to a raw, underexposed X-ray to mathematically expand the dark background regions, revealing the faint outer edges of the ribcage.
- **Power-Law Transformation (Gamma):** Apply a fractional power-law transformation ($\gamma < 1.0$) to fine-tune the midtones,

reducing the harsh contrast of the bones so the soft lung tissue becomes clearer.

Task 2: Multi-Modal Cardiac Image Fusion

The Scenario:

You are now a senior perception engineer at a cardiac research institute. Doctors frequently struggle to analyze the heart because CT scans show excellent structural boundaries, but MRI scans are vastly superior at showing soft tissue variations. Your objective is to mathematically fuse perfectly aligned CT and MRI slices of the same heart into a single, unified matrix to provide a comprehensive diagnostic view.

The Operations:

- **Load Modalities:** Load one CT slice and its perfectly corresponding MRI slice from the dataset.
- **Histogram Equalization:** Apply equalization to both the CT and MRI matrices independently to maximize their dynamic ranges before blending.
- **Color Mapping and Fusion:** Convert the enhanced CT and MRI images into colored heatmaps (e.g., mapping the MRI to a specific color map). Overlay the matrices to create a unified view.
- **Multi-Modal Weighted Fusion:** Use `cv2.addWeighted()` to adjust the alpha and beta blending values. Assign a heavier weight to the CT matrix to preserve the sharp anatomical edges, and a lighter weight to the MRI matrix to highlight the internal tissue variations.
- **Logarithmic and Power-Law Transformations:** Pass the final fused matrix through logarithmic and power-law functions to ensure no critical data is crushed into pure black or blown out to pure white during the addition process.
- **Comparative Analysis:** Display the standalone CT frame, the standalone MRI frame, and the fused output side-by-side to

mathematically and visually validate the success of the fusion.

Task 3: Real-Time Echocardiogram Video Analysis

The Scenario:

Moving from static images to dynamic data, your final task involves real-time processing of an Echocardiogram (ultrasound video of a beating heart). Ultrasound footage is notoriously noisy, murky, and low-contrast. You must build a real-time OpenCV pipeline that captures each frame of the .mp4 video, applies your mathematical enhancements, and displays the corrected feed live alongside the raw feed.

The Operations:

- **Video Capture Setup:** Initialize `cv2.VideoCapture()` to read from the provided ultrasound .mp4 file rather than a webcam.
- **Real-Time Video Processing Loop:** Start a `while` loop to extract frames continuously. For every individual frame matrix:
    - Apply Histogram Equalization to combat the murky ultrasound contrast.
    - Convert the frame to a colored heatmap (`COLORMAP_JET`) to highlight blood flow intensities.
    - Apply a color balance adjustment.
    - Apply a logarithmic transformation to reveal the darkest regions of the heart chambers.
    - Apply a power-law transformation to suppress the blinding white backscatter noise typical of ultrasound probes.
- **Monitoring Array:** Concatenate the frames and use `cv2.imshow()` to display the raw, unedited ultrasound stream side-by-side with the fully enhanced pipeline frames.

Required Resources:

For Task 1: Chest X-Rays

- **Dataset:** *COVID19+PNEUMONIA+NORMAL Chest X-Ray Image Dataset.*

- **Link:**[https://www.kaggle.com/datasets/sachinkumar413/covid-pneumonia-normal-chest-xray-images]

For Task 2: CT & MRI Fusion

- **Dataset:** *Heart CT & MRI Dataset.*

- **Link:**[https://www.kaggle.com/datasets/ziya07/heart-ct-and-mri-dataset]

For Task 3: Ultrasound Video Analysis

- **Dataset:** *Stanford EchoNet-Dynamic Dataset.*

- **Link:**[https://www.kaggle.com/datasets/manojkumarcs28/echonet-dynamic-by-stanford-university]